ADVANCED DATABASE TECHNIQUES

STORED PROCEDURES

OBJECTIVE

- Understand the concept and purpose of SQL stored procedures.
- Learn how to create, modify, and execute SQL stored procedures.
- Explore the advantages and best practices of using stored procedures in database management.
- Utilize stored procedures to enhance data processing, improve performance, and ensure data integrity within database systems.

STORED PROCEDURE

- Is a generic program unit that is executed on request and that can accept multiple input and output parameters.
- A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again.
- A set of Structured Query Language (SQL) statements with an assigned name, which are stored in a relational database management system (RDBMS) as a group, so it can be reused and shared by multiple programs.

Stored procedures can be used as a security layer. You can perform input validation in stored procedures, and you can use stored procedures to allow activities only if they make sense as a whole unit, as opposed to allowing users to perform activities directly against objects.

Stored procedures also give you the benefits of encapsulation; if you need to change the implementation of a stored procedure because you developed a more efficient way to achieve a task, you can issue an ALTER PROCEDURE statement.

ADVANTAGE OF USING STORED PROCEDURE

Reusable: Multiple users and applications can easily use and reuse stored procedures by merely calling it.

Easy to modify: You can quickly change the statements in a stored procedure as and when you want to, with the help of the ALTER TABLE command.

Security: Stored procedures allow you to enhance the security of an application or a database by restricting the users from direct access to the table.

Low network traffic: The server only passes the procedure name instead of the whole query, reducing network traffic.

Increases performance: Upon the first use, a plan for the stored procedure is created and stored in the buffer pool for quick execution for the next time.

Advantages	Disadvantages
It is faster.	It is difficult to debug.
It is pre-compiled.	Need expert developer, since difficult to write code.
It reduces network traffic.	It is database dependent.
It is reusable.	It is non-portable.
It's security is high .	It is expensive.

SYNTAX

```
DELIMITER $$

CREATE

    PROCEDURE `hr`.`procAddEmployee`()

        BEGIN

        END$$

DELIMITER ;
```

Procedure Name

STEPS TO CREATE A STORED PROCEDURE

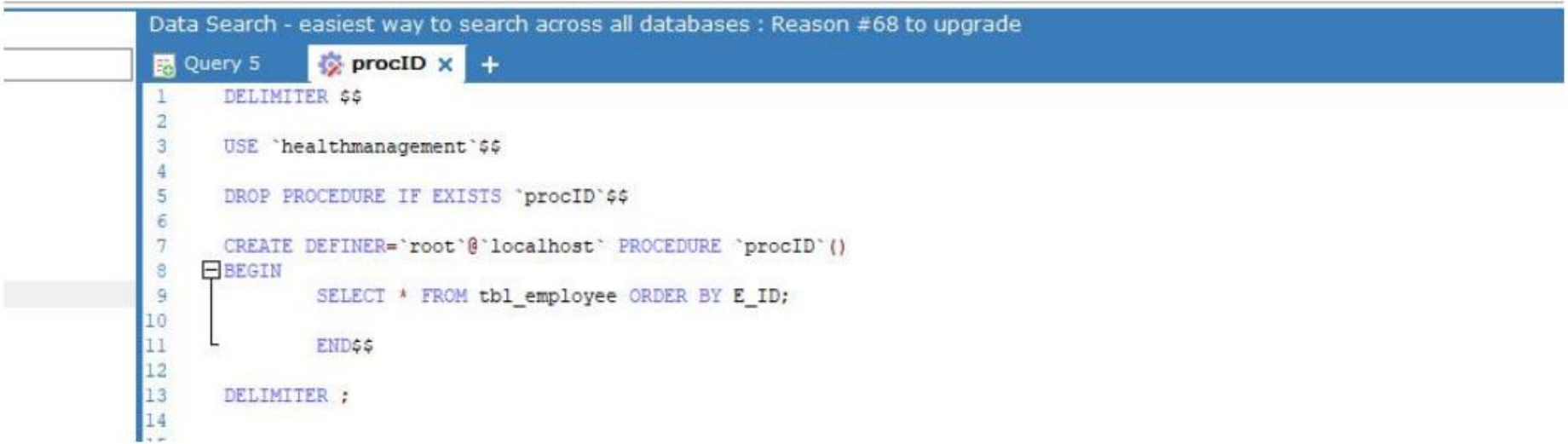
- Use DELIMITER command to avoid conflicts with semicolon inside the procedure.
- Create procedure statement, use CREATE PROCEDURE statement to define the stored procedure and followed by the name of the procedure.
- Define the parameter if needed.
- Use BEGIN and END keyword to define the block of SQL statement.
- Write the procedure body.
- Reset the delimiter.

Parameter of the procedure

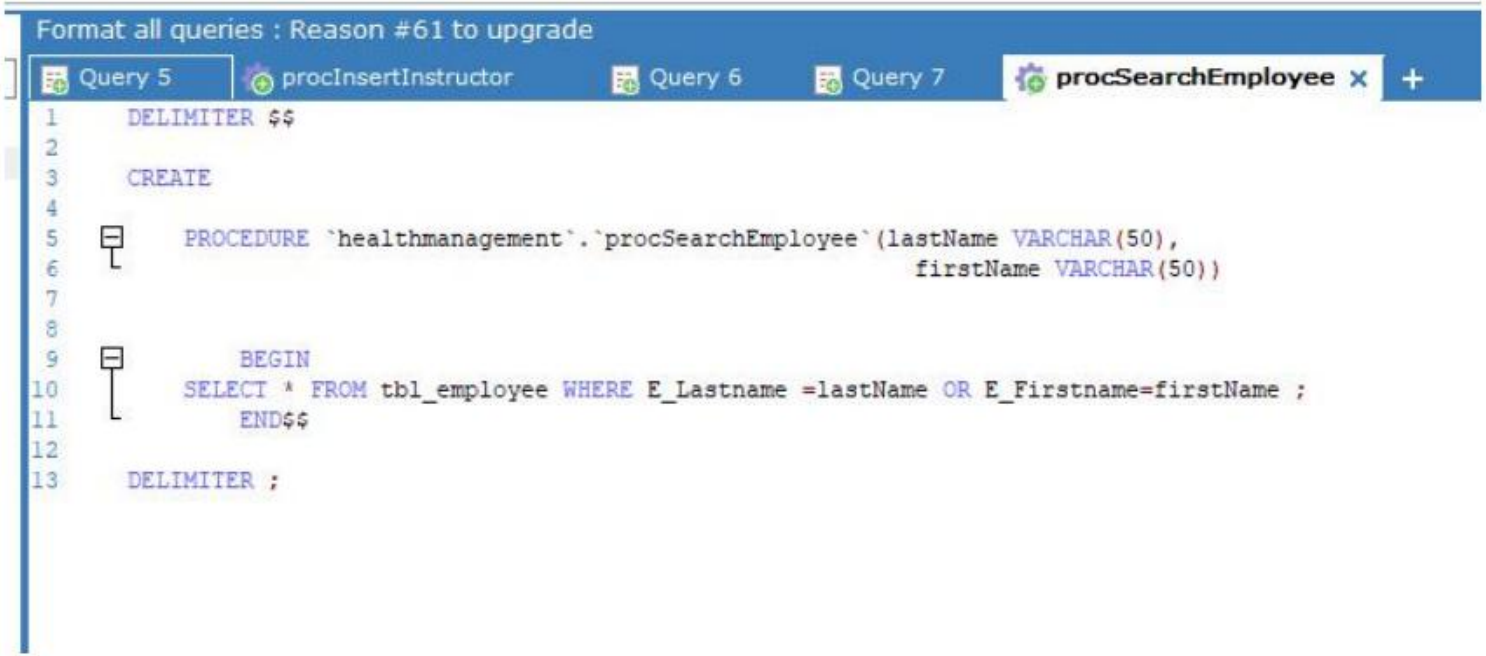
```
DELIMITER $$

USE `employee`$$

DROP PROCEDURE IF EXISTS `procInsertInstructor`$$

CREATE DEFINER=`root`@`localhost` PROCEDURE `procInsertInstructor`(p_id INT,p_InsID VARCHAR(20),
                                    p_fullName VARCHAR(20),p_address VARCHAR(20),p_con VARCHAR(20))
BEGIN
                INSERT INTO tbl_instructor(id,Ins_ID,Fullname, Address,ContactNo)
                VALUES(p_id,p_InsID,p_fullname,p_address,p_con);
        END$$

DELIMITER ;
```

Query

STORED PROCEDURE WITHOUT PARAMETER



STORED PROCEDURE WITH PARAMETER



No need to learn cryptic command line tools for backups : Reason #44 to upgrade

Query 5 | Query 6 | procUpdatePersonnel | procUpdateEmployee ×

```
1   DELIMITER $$
2
3   USE `healthmanagement`$$
4
5   DROP PROCEDURE IF EXISTS `procUpdateEmployee`$$
6
7   CREATE DEFINER=`root`@`localhost` PROCEDURE `procUpdateEmployee`(p_EID INT,
8                                                   p_LastName VARCHAR (50),
9                                                   p_FirstName VARCHAR(50),
10                                                  p_MiddleName VARCHAR(50),
11                                                  p_Bdate DATETIME,
12                                                  p_Gender VARCHAR(6),
13                                                  p_Status VARCHAR(10),
14                                                  p_Province VARCHAR(100),
15                                                  p_City VARCHAR(100),
16                                                  p_Barangay VARCHAR(100),
17                                                  p_Purok VARCHAR(100),
18                                                  p_Job VARCHAR(100),
19                                                  p_Contactnum VARCHAR(100),
20                                                  p_Age INTEGER
21                                                  )
```

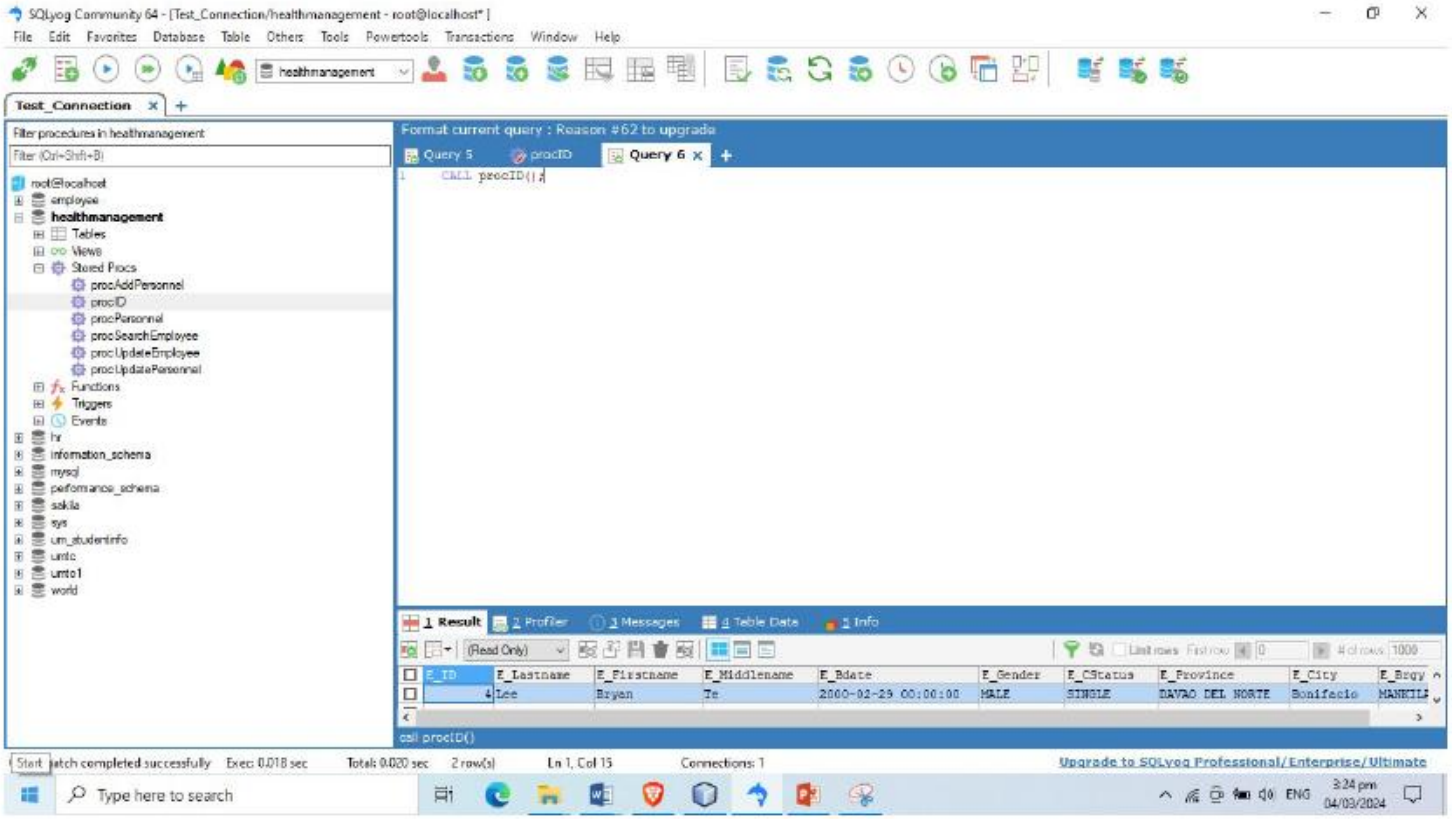
```
BEGIN
UPDATE tbl_employee
                SET
                                        E_Lastname=p_LastName,
                                        E_Firstname=p_FirstName,
                                        E_Middlename=p_MiddleName,
                                        E_Bdate=p_Bdate,
                                        E_Gender=p_Gender,
                                        E_CStatus=p_Status,
                                        E_Province=p_Province,
                                        E_City=p_City,
                                        E_Brgy=p_Barangay,
                                        E_Street=p_Purok,
                                        E_JobTitle=p_Job,
                                        E_Contact=p_Contactnum,
                                        E_Age=p_Age
                                        WHERE E_ID=p_EID;

        END$$

DELIMITER ;
```

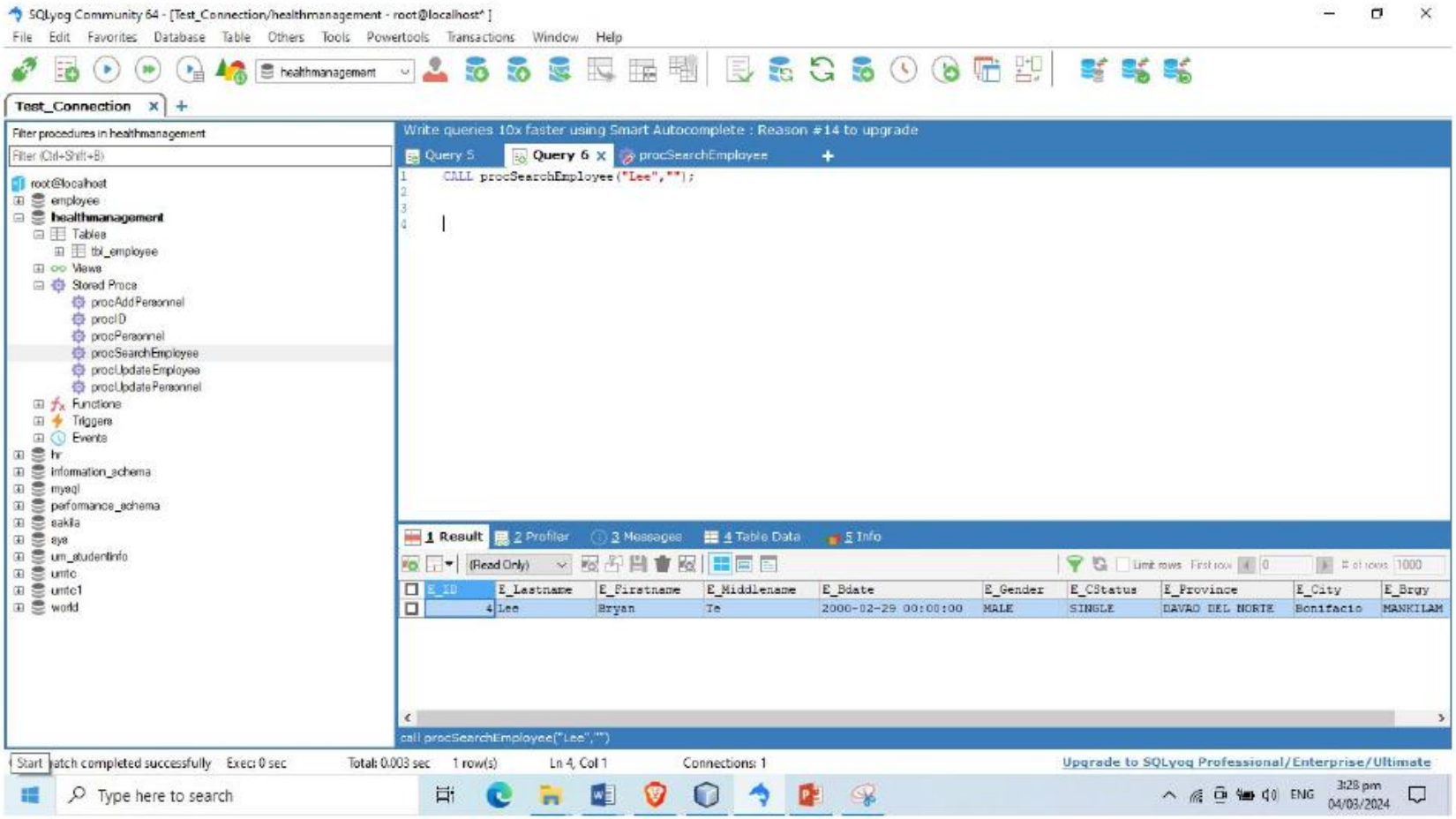
CONDITION IN STORED PROCEDURE

Query 5 | Query 6 | procUpdatePersonnel | procUpdateEmployee | procAreaCircle | procAgeClass

```
DELIMITER $$

USE `umtc`$$

DROP PROCEDURE IF EXISTS `procAgeClass`$$

CREATE DEFINER=`root`@`localhost` PROCEDURE `procAgeClass`(age INT)
BEGIN
    DECLARE age_group VARCHAR(10);

    IF age < 18 THEN
        SET age_group = 'Minor';
    ELSEIF age >= 18 AND age < 65 THEN
        SET age_group = 'Adult';
    ELSE
        SET age_group = 'Senior';
    END IF;

    SELECT age_group;
END$$

DELIMITER ;
```

CALLING OF STORED PROCEDURE




SQLyog Community 64 - [Test_Connection/healthmanagement - root@localhost*]
File Edit Favorites Database Table Others Tools Powertools Transactions Window Help
healthmanagement
Test_Connection
Filter procedures in healthmanagement
Filter (Ctrl+Shift+B)
root@localhost
employee
healthmanagement
Tables
tbl_employee
Views
Stored Proca
procAddPersonnel
procID
procPersonnel
procSearchEmployee
procUpdateEmployee
procUpdatePersonnel
Functions
Triggers
Events
hr
information_schema
myaql
performance_schema
sakila
sys
um_studentinfo
umtc
umtc1
world
Write queries 10x faster using Smart Autocomplete : Reason #14 to upgrade
Query 5
Query 6
procSearchEmployee
1 CALL procSearchEmployee("Lee","");
1 Result
2 Profiler
3 Messages
4 Table Data
5 Info
(Read Only)
E_ID E_Lastname E_Firstname E_Middlename E_Bdate E_Gender E_CStatus E_Province E_City E_Brgy
4 Lee Bryan Te 2000-02-29 00:00:00 MALE SINGLE DAVAO DEL NORTE Bonifacio MANKILAM
call procSearchEmployee("Lee","")
Start
atch completed successfully Exec: 0 sec Total: 0.003 sec 1 row(s) Ln 4, Col 1 Connections: 1
Upgrade to SQLyog Professional/Enterprise/Ultimate
Type here to search
ENG
3:28 pm
04/03/2024